

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №9
ДИСЦИПЛИНЫ
«ОСНОВЫ НЕЙРОННЫХ СЕТЕЙ»
ВАРИАНТ № 2**

Выполнил:
Левашев Тимур Рашидович
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии Воронкин Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема работы: «Изучение архитектуры автокодировщиков и их применение для сжатия данных, обучения признаков и восстановления изображений»

Цель работы: освоить принципы построения и обучения автокодировщиков (autoencoders) для решения различных задач обработки изображений, включая: сжатие данных через обучение латентному представлению, визуализацию скрытого пространства признаков, восстановление изображений после их искажения

Ссылка на Git-репозиторий: https://github.com/mazy99/neural-networks_9

Порядок выполнения работы

1 Построить автокодировщик с 2-мерным латентным пространством и визуализировать разделение классов цифр

1.1 Подключение необходимых для работы библиотек

Листинг 1.1 – Подключение библиотек

```
# Работа с операционной системой
import os

# Отрисовка графиков
import matplotlib.pyplot as plt

# Операции с путями
import glob

# Работа с массивами данных
import numpy as np

# Слои
from tensorflow.keras.layers import Dense, Flatten, Reshape, Input,
Conv2DTranspose, concatenate, Activation, MaxPooling2D, Conv2D,
BatchNormalization, Concatenate

# Модель
from tensorflow.keras import Model
```

```

# Загрузка модели
from tensorflow.keras.models import load_model

# Датасет
from tensorflow.keras.datasets import mnist

# Оптимизатор для обучения модели
from tensorflow.keras.optimizers import Adam

# Коллбэки для выдачи информации в процессе обучения
from tensorflow.keras.callbacks import LambdaCallback

%matplotlib inline

```

1.2 Создание дополнительных утилит (отрисовка объектов в скрытом пространстве и удаление изображений при обучении новой модели)

Листинг 1.2 – Дополнительные утилиты

```

def ae_on_epoch_end(epoch, logs):
    print('_____')
    print(f'*** ЭПОХА: {epoch+1}, loss: {logs["loss"]} ***')
    print('_____')

    plt.figure(dpi=100)

    predict = encoder.predict(X_train)

    scatter = plt.scatter(predict[:,0],predict[:,1], c=y_train, alpha=0.6, s=5)

    legend2 = plt.legend(*scatter.legend_elements(), loc='upper right',
title='Классы')

    paths = glob.glob('*.jpg')
    plt.savefig(f'image_{str(len(paths))}.jpg')

    plt.show()

ae_callback = LambdaCallback(on_epoch_end=ae_on_epoch_end)

```

```
def clean():

    paths = glob.glob('*.jpg')

    for p in paths:
        os.remove(p)

clean()
```

1.3 Загрузка данных в массивы и разделение их на обучающую и тестовую выборки

Листинг 1.2 – Загрузка и разделение данных

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
print(f'X_train shape: {X_train.shape}')
print(f'y_train shape: {y_train.shape}')
print(f'X_test shape: {X_test.shape}')
print(f'y_test shape: {y_test.shape}')
```

```
X_train shape: (60000, 28, 28)
y_train shape: (60000,)
X_test shape: (10000, 28, 28)
y_test shape: (10000,)
```

Рисунок 1.1 – Размерности массивов

1.4 Нормализация и фильтрация данных

Листинг 1.3 – Нормализация данных и добавление канала

```
X_train = X_train.astype('float32')/255.
X_train = X_train.reshape(-1, 28, 28, 1)

print(f'X_train shape: {X_train.shape}')
```

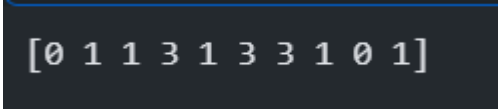
```
X_train shape: (18796, 28, 28, 1)
```

Рисунок 1.2 – Размерность обучающего массива после нормализации

Листинг 1.4 – Фильтрация данных

```
numbers = [0, 1, 3]
mask = np.array([(i in numbers) for i in y_train])
X_train = X_train[mask]
y_train = y_train[mask]

print(y_train[:10])
```



[0 1 1 3 1 3 3 1 0 1]

Рисунок 1.3 – Результат фильтрации

1.5 Создание и обучение модели нейронной сети

Листинг 1.5 – Создание энкодера

```
input_img = Input(shape=(28, 28, 1))

x = Conv2D(32, (3, 3), padding='same', activation='relu')(input_img)
x = BatchNormalization()(x)
x = MaxPooling2D((2, 2), padding='same')(x)

x = Conv2D(64, (3, 3), padding='same', activation='relu')(x)
x = BatchNormalization()(x)
x = MaxPooling2D((2, 2), padding='same')(x)

x = Flatten()(x)
x = Dense(128, activation='relu')(x)

latent = Dense(2, name='latent_space')(x)

encoder = Model(input_img, latent, name='encoder')
```

Листинг 1.6 – Создание декодера

```
autoencoder_input = input_img
encoded = encoder(autoencoder_input)
decoded = decoder(encoded)
```

```

autoencoder = Model(autoencoder_input, decoded, name='autoencoder')

autoencoder.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='mse'
)

autoencoder.summary()

```

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 28, 28, 1)	0
encoder (Functional)	(None, 2)	420,994
decoder (Functional)	(None, 28, 28, 1)	65,473

Рисунок 1.4 – Структура модели

Листинг 1.7 – Обучение модели

```

history = autoencoder.fit(
    X_train,
    X_train,
    epochs=12,
    batch_size=128,
    shuffle=True,
    callbacks=[ae_callback]
)

```

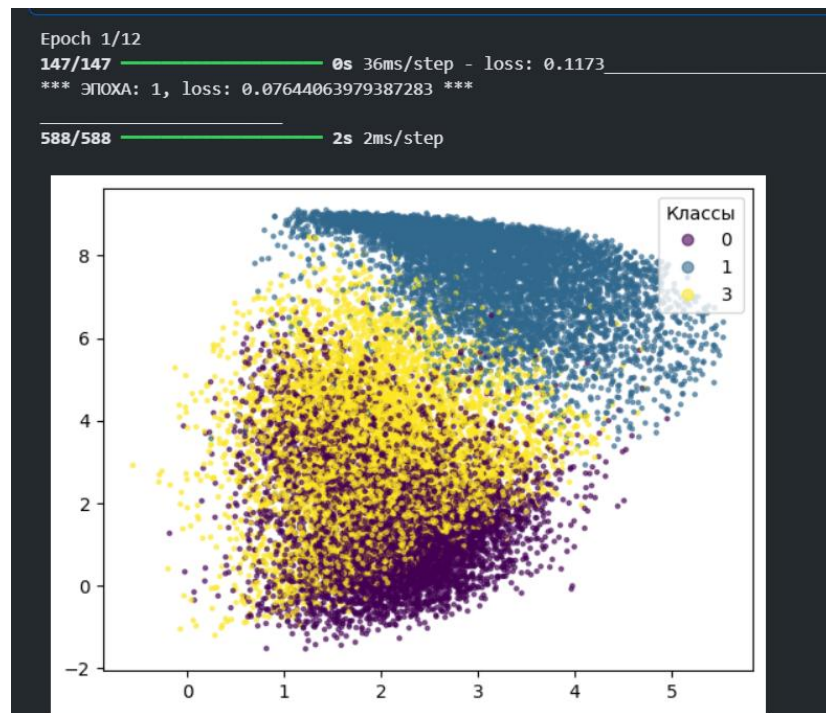


Рисунок 1.5 – Первая эпоха обучения модели

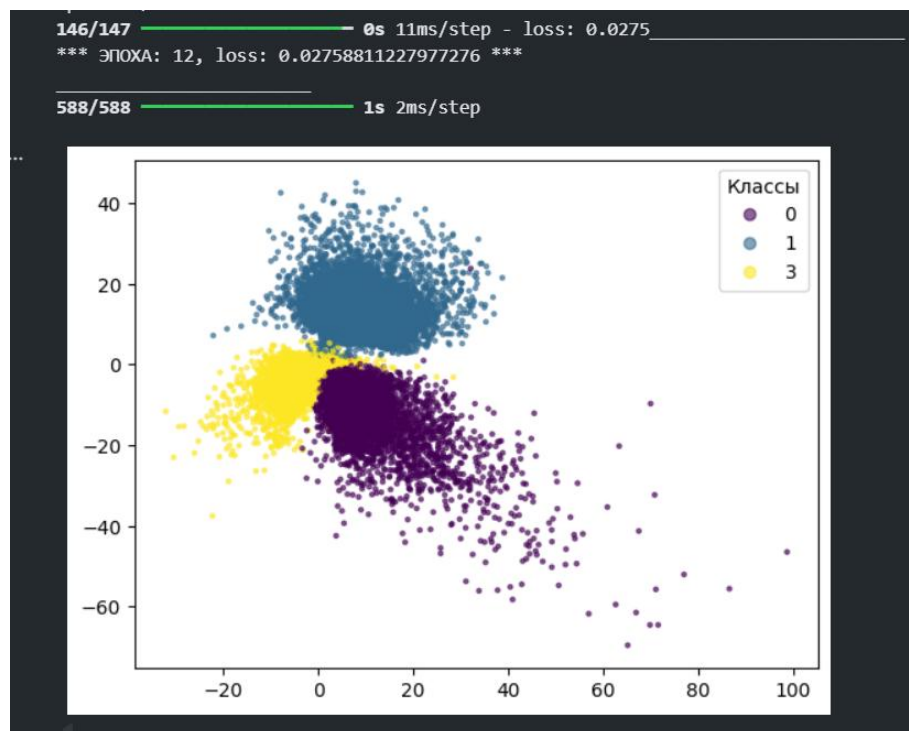


Рисунок 1.6 – Последняя эпоха обучения модели

1.6 Визуализация реконструкций модели

Листинг 1.8 – Визуализация реконструкций

```
predictions = autoencoder.predict(X_train)

plt.figure(figsize=(20, 6))
n = 10
for i in range(n):
    idx = np.random.randint(0, len(X_train))

    plt.subplot(2, n, i+1)
    plt.imshow(X_train[idx].squeeze(), cmap='gray')
    plt.title(f'Цифра: {y_train[idx]}')
    plt.axis('off')

    plt.subplot(2, n, i+1+n)
    plt.imshow(predictions[idx].squeeze(), cmap='gray')
    plt.axis('off')
plt.tight_layout()
plt.show()
```



Рисунок 1.7 – Визуализация реконструкций

2 Обучить автокодировщик для восстановления изображений цифры 5

2.1 Импорт необходимых для работы библиотек

Листинг 2.1 – Импорт библиотек

```
# Для операций с тензорами
import numpy as np
```

```

# Для отрисовки
import matplotlib.pyplot as plt

# Для создания модели
from tensorflow.keras.models import Model

# Необходимые слои
from tensorflow.keras.layers import Input, Conv2DTranspose, MaxPooling2D, Conv2D,
BatchNormalization

# Слои для латентного пространства модели
from tensorflow.keras.layers import Flatten, Reshape, Dense

# Оптимизатор
from tensorflow.keras.optimizers import Adam

# Для загрузки базы
from tensorflow.keras.datasets import mnist

```

2.2 Загрузка и нормализация данных

Листинг 2.2 – Загрузка и нормализация данных

```

(X_train, y_train), (X_test, y_test) = mnist.load_data()

X_train = X_train.astype('float32')/255.
X_test = X_test.astype('float32')/255.

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")

```

```

X_train shape: (60000, 28, 28)
X_test shape: (10000, 28, 28)

```

Рисунок 2.1 – Размерности обучающей и тестовой выборки

2.3 Приведение данных к форме удобной для обучения в Keras

Листинг 2.3 – Приведение данных к удобной форме

```

X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

```

```
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
```

```
X_train shape: (60000, 28, 28, 1)
X_test shape: (10000, 28, 28, 1)
```

Рисунок 2.2 – Результат приведения данных

2.4 Отбор красивых пятерок

Листинг 2.4 – Отбор пятерок

```
mask = y_train == 5
X_train = X_train[mask]
y_train = y_train[mask]

print(y_train)
print(y_train.shape)

mask = y_test == 5
X_test = X_test[mask]
y_test = y_test[mask]

print(y_test)
print(y_test.shape)
```

```
print(y_train)
print(y_train.shape)
```

```
mask = y_test == 5
X_test = X_test[mask]
y_test = y_test[mask]
```

```
print(y_test)
print(y_test.shape)
```

[illegible]

Рисунок 2.3 – Результат отбора пятёрок на тестовой выборке

Листинг 2.5 – Визуализация набора данных

```
plt.figure(figsize=(15, 15))

for i in range(100):
    plt.subplot(10, 10, i + 1)
    plt.imshow(X_train[i].reshape(28, 28), cmap='gray')
    plt.title(str(i))
    plt.axis('off')

plt.show()
```

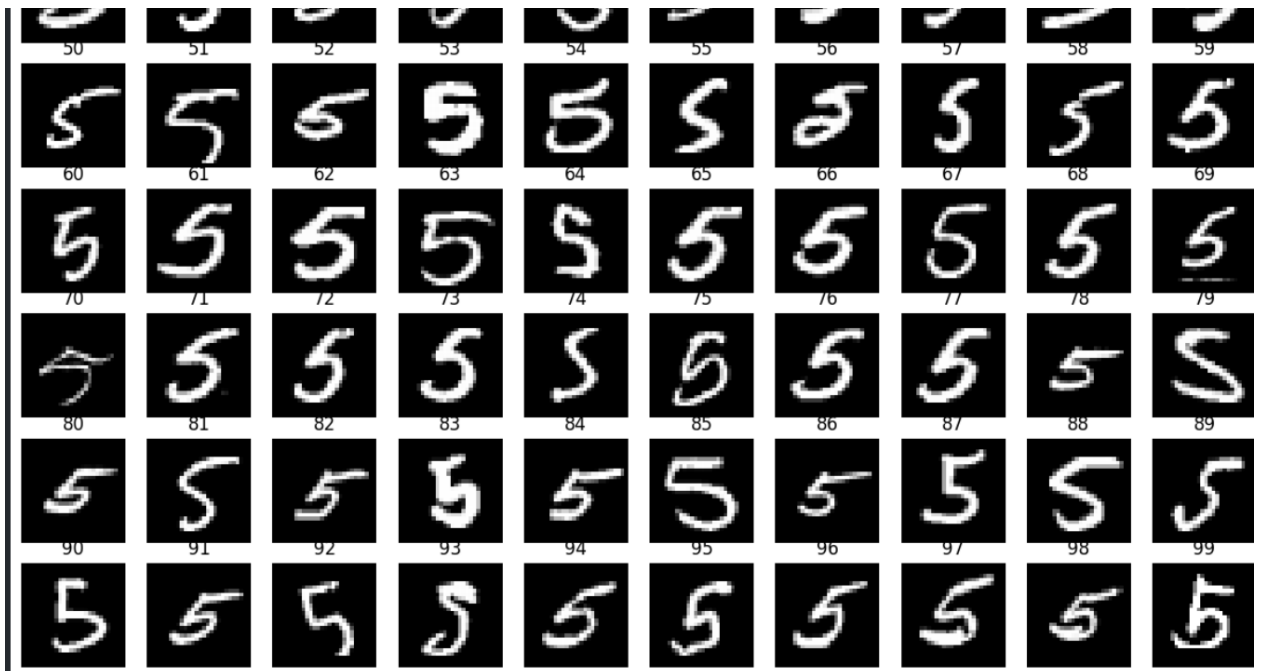


Рисунок 2.4 – Визуализация набора данных

Листинг 2.6 – Отбор красивых пятёрок

```
beautiful_idx = [0, 2, 3, 8, 63, 91, 40, 23, 87, 47]

best_fives = X_train[beautiful_idx]

plt.figure(figsize=(12, 3))

for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(best_fives[i].reshape(28, 28), cmap='gray')
    plt.axis('off')

plt.show()
```



Рисунок 2.5 – Результат отбора

2.5 Создание и обучение модели

Листинг 2.7 – Создание модели

```
input_img = Input(shape=(28, 28, 1))

# Encoder слой
x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = BatchNormalization()(x)
x = MaxPooling2D((2, 2), padding='same')(x)

x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
x = BatchNormalization()(x)
x = MaxPooling2D((2, 2), padding='same')(x)

x = Flatten()(x)
latent = Dense(64, activation='relu')(x)

# Decoder слой
x = Dense(7 * 7 * 64, activation='relu')(latent)
x = Reshape((7, 7, 64))(x)

x = Conv2DTranspose(64, (3, 3), strides=2, activation='relu', padding='same')(x)
x = BatchNormalization()(x)

x = Conv2DTranspose(32, (3, 3), strides=2, activation='relu', padding='same')(x)
x = BatchNormalization()(x)

output_img = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

autoencoder = Model(input_img, output_img)

autoencoder.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='mse'
)

autoencoder.summary()
```

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 28, 28, 1)	0
conv2d_9 (Conv2D)	(None, 28, 28, 32)	320
batch_normalization_12 (BatchNormalization)	(None, 28, 28, 32)	128
max_pooling2d_6 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_10 (Conv2D)	(None, 14, 14, 64)	18,496
batch_normalization_13 (BatchNormalization)	(None, 14, 14, 64)	256
max_pooling2d_7 (MaxPooling2D)	(None, 7, 7, 64)	0
flatten_3 (Flatten)	(None, 3136)	0
dense_6 (Dense)	(None, 64)	200,768
dense_7 (Dense)	(None, 3136)	203,840
reshape_3 (Reshape)	(None, 7, 7, 64)	0
conv2d_transpose_6 (Conv2DTranspose)	(None, 14, 14, 64)	36,928
batch_normalization_14 (BatchNormalization)	(None, 14, 14, 64)	256
conv2d_transpose_7 (Conv2DTranspose)	(None, 28, 28, 32)	18,464
batch_normalization_15 (BatchNormalization)	(None, 28, 28, 32)	128
conv2d_11 (Conv2D)	(None, 28, 28, 1)	289

Рисунок 2.6 – Структура модели

Листинг 2.8 – Обучение модели

```

history = autoencoder.fit(
    X_train,
    Y_train,
    epochs=30,
    batch_size=128,
    validation_data=(X_test,Y_test),
    shuffle=True
)

```

```

Epoch 2/30
43/43 ————— 0s 11ms/step - loss: 0.0785 - val_loss: 0.0945
Epoch 3/30
43/43 ————— 0s 11ms/step - loss: 0.0768 - val_loss: 0.0935
Epoch 4/30
43/43 ————— 0s 11ms/step - loss: 0.0764 - val_loss: 0.0954
Epoch 5/30
43/43 ————— 1s 12ms/step - loss: 0.0756 - val_loss: 0.0970
Epoch 6/30
43/43 ————— 1s 13ms/step - loss: 0.0752 - val_loss: 0.0986
Epoch 7/30
43/43 ————— 1s 19ms/step - loss: 0.0746 - val_loss: 0.1003
Epoch 8/30
43/43 ————— 1s 15ms/step - loss: 0.0742 - val_loss: 0.1000
Epoch 9/30
43/43 ————— 1s 11ms/step - loss: 0.0740 - val_loss: 0.0988
Epoch 10/30
43/43 ————— 1s 11ms/step - loss: 0.0730 - val_loss: 0.0989
Epoch 11/30
43/43 ————— 1s 11ms/step - loss: 0.0697 - val_loss: 0.0906
Epoch 12/30
43/43 ————— 0s 11ms/step - loss: 0.0653 - val_loss: 0.0925
Epoch 13/30
...
Epoch 29/30
43/43 ————— 1s 13ms/step - loss: 0.0092 - val_loss: 0.0837
Epoch 30/30
43/43 ————— 1s 15ms/step - loss: 0.0077 - val_loss: 0.0840

```

Рисунок 2.7 – Результат обучения модели

Final MSE: 0.007652913220226765

Рисунок 2.8 – Финальный результат MSE

2.6 Визуализация пятерок из тестовой выборки после автокодировщика

Листинг 2.9 – Визуализация пятерок

```

test_pred = autoencoder.predict(X_test, verbose=0)

plt.figure(figsize=(20, 10))
n = 10

for i in range(n):
    idx = np.random.randint(0, len(X_test))

    plt.subplot(3, n, i+1)
    plt.imshow(X_test[idx].squeeze(), cmap='gray')
    plt.title(f'Оригинал')
    plt.axis('off')

plt.subplot(3, n, i+1+n)

```

```
plt.imshow(test_pred[idx].squeeze(), cmap='gray')
plt.title(f'Декодированный')
plt.axis('off')

plt.suptitle('Пятерки из тестовой выборки после автокодировщика', fontsize=14)
plt.tight_layout()
plt.show()
```

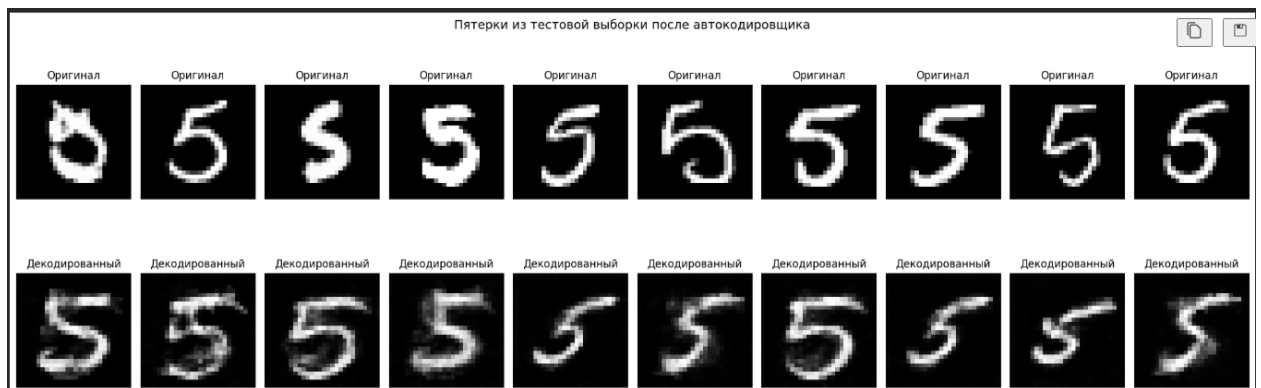


Рисунок 2.9 – Результат работы автокодировщика

3 Научить автокодировщик восстанавливать изображения, поврежденные черными квадратами

3.1 Загрузка и нормализация данных, а так же приведение их к удобному для работы с keras виду

Листинг 3.1 – Загрузка данных

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 — 0s 0us/step
```

Рисунок 3.1 – Результат загрузки данных

Листинг 3.2 – Нормализация данных

```
X_train = X_train.astype('float32')/255.
X_test = X_test.astype('float32')/255.
```

Листинг 3.3 – Приведение к удобному для работы с keras виду

```
X_train = X_train.reshape((-1, 28, 28, 1))
X_test = X_test.reshape((-1, 28, 28, 1))
```

Листинг 3.4 – Зашумление изображений

```
def add_black_square(images, square_size=8):
    noisy_images = images.copy()

    for img in noisy_images:
        x = np.random.randint(0, 28 - square_size)
        y = np.random.randint(0, 28 - square_size)

        img[x:x + square_size, y:y + square_size, :] = 0

    return noisy_images

X_train_noisy = add_black_square(X_train)
X_test_noisy = add_black_square(X_test)

print("X_train_noisy:", X_train_noisy.shape)
print("X_train:", X_train.shape)
print("X_test_noisy:", X_test_noisy.shape)
print("X_test:", X_test.shape)
```

```
X_train_noisy: (60000, 28, 28, 1)
X_train: (60000, 28, 28, 1)
X_test_noisy: (10000, 28, 28, 1)
X_test: (10000, 28, 28, 1)
```

Рисунок 3.2 – Размерности массивов

3.2 Создание и обучение модели

Листинг 3.5 – Создание модели

```
input_img = Input(shape=(28, 28, 1))

# Encoder
x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = BatchNormalization()(x)
```

```

x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = MaxPooling2D((2, 2), padding='same')(x)

x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
x = BatchNormalization()(x)
x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
x = MaxPooling2D((2, 2), padding='same')(x)

# Decoder
x = Conv2DTranspose(64, (3, 3), strides=2, activation='relu', padding='same')(x)
x = BatchNormalization()(x)

x = Conv2DTranspose(32, (3, 3), strides=2, activation='relu', padding='same')(x)
x = BatchNormalization()(x)

output_img = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

autoencoder = Model(input_img, output_img)

autoencoder.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='mse'
)

autoencoder.summary()

```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 28, 28, 32)	320
batch_normalization (BatchNormalization)	(None, 28, 28, 32)	128
conv2d_1 (Conv2D)	(None, 28, 28, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_2 (Conv2D)	(None, 14, 14, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 14, 14, 64)	256
conv2d_3 (Conv2D)	(None, 14, 14, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 64)	36,928
batch_normalization_2 (BatchNormalization)	(None, 14, 14, 64)	256
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)	18,464
batch_normalization_3 (BatchNormalization)	(None, 28, 28, 32)	128
conv2d_4 (Conv2D)	(None, 28, 28, 1)	289

Рисунок 3.3 – Структура модели

Листинг 3.6 – Обучение модели

```
history = autoencoder.fit(  
    X_train_noisy,  
    X_train,  
    epochs=20,  
    batch_size=128,  
    shuffle=True,  
    validation_data=(X_test_noisy, X_test)  
)
```

```
Epoch 1/20  
469/469 ————— 20s 25ms/step - loss: 0.0178 - val_loss: 0.0321  
Epoch 2/20  
469/469 ————— 6s 13ms/step - loss: 0.0065 - val_loss: 0.0060  
Epoch 3/20  
469/469 ————— 6s 13ms/step - loss: 0.0054 - val_loss: 0.0053  
Epoch 4/20  
469/469 ————— 6s 13ms/step - loss: 0.0050 - val_loss: 0.0051  
Epoch 5/20  
469/469 ————— 6s 14ms/step - loss: 0.0047 - val_loss: 0.0050  
Epoch 6/20  
469/469 ————— 6s 13ms/step - loss: 0.0044 - val_loss: 0.0045  
Epoch 7/20  
469/469 ————— 6s 14ms/step - loss: 0.0043 - val_loss: 0.0044  
Epoch 8/20  
469/469 ————— 6s 14ms/step - loss: 0.0041 - val_loss: 0.0044  
Epoch 9/20  
469/469 ————— 6s 14ms/step - loss: 0.0041 - val_loss: 0.0047  
Epoch 10/20  
469/469 ————— 6s 14ms/step - loss: 0.0039 - val_loss: 0.0043  
Epoch 11/20  
469/469 ————— 7s 14ms/step - loss: 0.0039 - val_loss: 0.0042  
Epoch 12/20  
469/469 ————— 7s 14ms/step - loss: 0.0038 - val_loss: 0.0043  
Epoch 13/20  
...  
Epoch 19/20  
469/469 ————— 7s 14ms/step - loss: 0.0034 - val_loss: 0.0040  
Epoch 20/20  
469/469 ————— 7s 15ms/step - loss: 0.0034 - val_loss: 0.0041
```

Рисунок 3.4 – Результат обучения модели

```
313/313 ————— 1s 3ms/step - loss: 0.0041  
Test MSE: 0.004087679088115692  
Требуемая ошибка достигнута
```

Рисунок 3.5 – Финальный результат MSE

3.3 Визуализация изображений после автокодировщика

Листинг 3.7 – Визуализация изображений

```
decoded_imgs = autoencoder.predict(X_test_noisy[:10])

plt.figure(figsize=(20, 6))

for i in range(10):
    # Зашумленное изображение
    plt.subplot(3, 10, i + 1)
    plt.imshow(X_test_noisy[i].reshape(28, 28), cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    # Восстановленное изображение
    plt.subplot(3, 10, i + 11)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Decoded")
    plt.axis('off')

    # Оригинал
    plt.subplot(3, 10, i + 21)
    plt.imshow(X_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

plt.show()
```

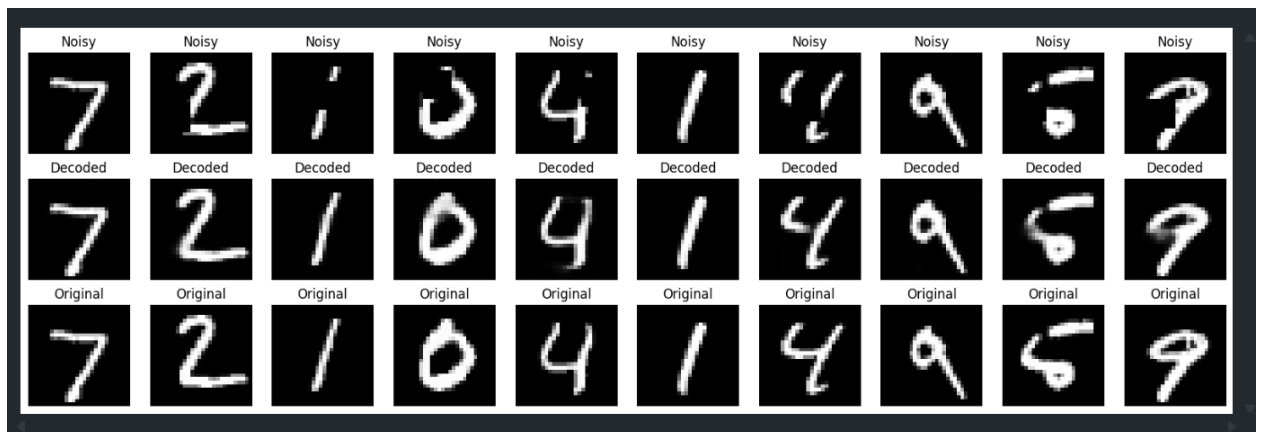


Рисунок 3.6 – Визуализация изображений

4 Выводы

В ходе выполнения работы были получены принципы построения и обучения автокодировщиков (autoencoders) для решения различных задач обработки изображений, включая: сжатие данных через обучение латентному представлению, визуализацию скрытого пространства признаков, восстановление изображений после их искажения